VIBE CONTROL

Version control for vibe coders

Auto-tracks your edit history as you code. No staging. No commit messages required. Just flow.

Version 1.0.1 · Created by Pratham Kumar Uikey

This is the complete guide to **VBC — Vibe Control**: what it is, why it exists, and everything you can do with it. For installation, see `documentation.md` in this repository. For a quick overview, see `README.md`.

What is VBC

Traditional version control asks you to decide, in advance, which moment is worth remembering. When you're actively working something out — trying an approach, backing out of it, trying another — that decision gets in the way. VBC removes it: start the watcher, and **every save becomes part of your project's history automatically** — no staging, no commit message required. When something important works, you mark it explicitly. When you hit a stable point, you drop a checkpoint — a byte-exact, restorable snapshot of the entire project, images and binaries included.

```
┌─ AUTO (watcher) ──────────────────────────────────────────┐
│  Every save  →  MF+  mf+  CF  MF-   (march flags)         │
└───────────────────────────────────────────────────────────┘

┌─ MANUAL (you decide) ─────────────────────────────────────┐
│  vbc plot we flag        →  WF   (file committed)         │
│  vbc plot us flag        →  UF   (all files committed)    │
│  vbc plot ckp as ckp<n>  →  CKP  (full project snapshot)  │
└───────────────────────────────────────────────────────────┘
```

VBC is not a replacement for Git. Use VBC *during* your coding session to capture every step. Push to Git when you're ready to ship.

What's New in 1.0.1

Installation — no Node.js required on Windows or Linux

Previous versions required Node.js on the target machine even for the Windows installer. VBC now ships as a standalone compiled binary with Node bundled directly in.

Checkpoints, reset, and flags are now byte-exact for every file type

Earlier builds read every flagged/checkpointed file as UTF-8 text before storing it — a lossy operation for anything that isn't plain text. Images, PDFs, .docx files, and executables were silently corrupted the moment they were flagged, checkpointed, or restored. vbc plot, vbc reset, and vbc clone now all round-trip binary content exactly as it was, for march flags, WF/UF, and checkpoints alike. Storage also got dramatically smaller: identical file content across many checkpoints is now stored once, not once per checkpoint.

Checkpoint numbering is now explicit

vbc plot ckp now requires you to choose the checkpoint number yourself:

```
vbc plot ckp as ckp1 -ms "v1.0 stable"
```

Unlike WF/UF (which still auto-increment), checkpoint numbers are yours to assign — they don't need to be sequential, but each one must be unique. This is deliberate: it lets you leave gaps for major milestones (e.g. jump straight to ckp10 for a big release) instead of being locked into a strict counter.

vbc clone — full control over what gets exported

vbc clone was rebuilt around a simple idea: checkpoints now capture everything (only .git/ and .vbc/ are ever excluded), and you decide what to export at clone time, not at checkpoint time.

```
vbc clone ckp1 as v1.zip                    # default: respects .vbcignore + .gitignore
vbc clone ckp1 as v1-full.zip --include all  # everything, no exclusions
vbc clone ckp1 as v1.zip --include .vbcignore  # also include .vbcignore-matched files
vbc clone ckp1 as v1.zip --include .gitignore  # also include .gitignore-matched files
vbc clone . as snapshot-now.zip              # zip the live working directory, right now
```

.vbcignore now also respects .gitignore patterns, and VBC's own codebase-dump exports (*_c3_*, *_scc_*) are excluded from tracking by default so a project never ends up checkpointing a full copy of itself.

WSL sync, rebuilt

Three commands handle every Windows ↔ WSL scenario in one place:

```
vbc sync wsl    # copy + link, in one step (most people want this)
vbc copy wsl    # push this Windows install into WSL
vbc global wsl  # (re)link vbc and add it to PATH inside WSL
```

If more than one WSL distro is registered, an arrow-key picker asks which one(s) to target — nothing is assumed, and Docker Desktop's internal utility distros are never listed as options.

VS Code extension fixes

The extension's diff view, checkpoint creation prompt, and "reset to checkpoint" action are all now correctly wired against the checkpoint numbering and storage changes above. The install-path lookup was also updated to find the new standalone binary on Windows automatically.

More reliable removal on Windows

vbc remove now stops a running watcher before deleting .vbc/ — an active file-watch handle could otherwise block the delete on Windows and leave .vbc/ partially removed.

Binary installers now install the VS Code extension too

Every binary install script picks up the extension package sitting next to it and installs it automatically if VS Code is found — the same behavior the full installer and .deb package already had, now consistent across every install method.

Quick Start

```
# 1. Go to your project
cd my-project

# 2. Initialize VBC (creates .vbc/ + .vbcignore)
vbc init

# 3. Start the watcher
vbc watch

# 4. Code normally — save files — flags appear automatically
#    → MF1+  main.py was fully replaced
#    → mf2+  utils.py had a partial edit

# 5. Commit a file when it is working
vbc plot we flag src/main.py -ms "login done"
```

```
# 6. Commit everything
vbc plot us flag -ms "session 1 complete"

# 7. Drop a stable checkpoint — you choose the number
vbc plot ckp as ckp1 -ms "v1.0 — auth + dashboard"
```

Flag Visual Language

Every change in VBC is recorded as a typed, colored flag.

Symbol	Color	Flag	Triggered	Meaning
●	Yellow	MF<n>+	Auto	Full file replaced — first save or >75% lines changed
○	Amber	mf<n>+	Auto	Partial edit — some lines changed
◆	Purple	CF<n>	Auto	Copy event — content matches another tracked file
◂	Red	MF<n>-	Auto	Revert — file shortened back toward earlier state
✓	Green	WF<n>	You	Single file commit — collapses all march flags for that file
✓	Blue	UF<n>	You	Project-wide commit — collapses march flags on all files
⬟	Gold	ckp<n>	You	Full project snapshot — number chosen by you, not auto-incremented

Flag numbering is per-file, per-type for march/WF/UF flags. `MF3+` = third full-replace on that file. `WF2` = second time you committed it. **Checkpoint numbers are project-wide and chosen by you.**

Full Command Reference

vbc init

Initialize a VBC repository in the current folder.

```
vbc init
```

Creates .vbc/ with header.json, snapshot.prt, stver.prt, and a default .vbcignore in the project root.

vbc watch

Start the file watcher. Auto-assigns march flags on every save.

```
vbc watch            # foreground — Ctrl+C to stop
vbc watch --bg       # background daemon
vbc watch --stop     # stop the background daemon
vbc watch --status   # check if daemon is running
vbc watch --tail     # live log tail of background daemon
```

vbc flag

Manually set a march flag without the watcher running.

```
vbc flag mf+ <file>
vbc flag mf+ <file> -ms "added helper"
```

```
vbc flag mf- <file>
vbc flag cf <file>
```

vbc plot

Set commit-level flags or a checkpoint.

```
# Commit a single file → WF (collapses all march flags for that file)
vbc plot we flag <file>
vbc plot we flag src/auth.py -ms "JWT login working"

# Commit all files → UF on each
vbc plot us flag
vbc plot us flag -ms "all routes done"

# Full project snapshot → CKP — you choose the number
vbc plot ckp as ckp1
vbc plot ckp as ckp2 -ms "v1.0 — login + dashboard complete"
```

On WE flag: march flags for that file (MF+, mf+, CF, MF-) are purged and collapsed into a single WF. Terminal shows what was collapsed.

On CKP: every file in the project is snapshotted — only .git/ and .vbc/ are excluded, everything else (including .vbcignore/.gitignore-matched files) is captured so vbc clone's --include flags have something real to work with. Content is stored byte-exact via the blob store, so images, PDFs, and binaries checkpoint correctly.

vbc reset

Restore files to a previous committed state.

```
# Restore file to its latest WF
vbc reset we flag <file>

# Restore file to a specific WF number
vbc reset we flag src/api.py -at WF2

# Restore all files to latest UF
vbc reset us flag

# Restore all files to a specific UF
vbc reset us flag -at UF3

# Restore the entire project to a checkpoint
vbc reset ckp -at ckp1
```

A backup of the current state is always taken before a reset is applied, so resets are never destructive.

vbc clone

Export a checkpoint — or the live working directory — as a zip archive.

```
vbc clone ckp1 as v1.zip                        # default: respects .vbcignore + .gitignore
vbc clone ckp1 as v1.zip --include .vbcignore    # also include .vbcignore-matched files
vbc clone ckp1 as v1.zip --include .gitignore    # also include .gitignore-matched files
vbc clone ckp1 as v1-full.zip --include all      # everything, no exclusions
vbc clone . as snapshot-now.zip                  # the live working directory, right now
```

`vbc clone .` doesn't need a checkpoint at all — it zips whatever's on disk at the moment you run it, exactly as-is (only .git/ and .vbc/ excluded).

vbc reconstruct

Restore a project from a cloned zip.

```
vbc reconstruct v1.zip as project-v1
vbc reconstruct project-march-backup.zip as restored
```

vbc construct

Scaffold a project from a .prt tree diagram file.

```
vbc construct myapp.prt as myapp        # create directory structure
vbc construct myapp.prt as myapp c3     # + generate codebase doc
vbc construct myapp.prt as myapp scc    # + run source code analysis
```

Example .prt file:

```
myapp/
├── src/
│   ├── main.py
│   └── utils.py
├── tests/
│   └── test_main.py
└── README.md
```

vbc log

Show flag history.

```
vbc log               # all files
vbc log src/main.py   # specific file
```

vbc status

Show repo stats — version counter, file count, flag counts, watcher state.

```
vbc status
```

vbc remove

Permanently delete a project's .vbc/ — all flags, checkpoints, and history. Source files and .vbcignore are left untouched.

```
vbc remove
vbc remove --force   # skip the confirmation prompt (aliases: -f, -y)
vbc rm               # alias for vbc remove
```

Stops a running watcher first if one is active, so an open file-watch handle can't block the delete. Run vbc init afterward to start tracking again from scratch.

vbc update

Update the CLI from a newly extracted folder (source-based installs).

```
vbc update C:\Downloads\vbc_unified   # Windows
vbc update /home/user/vbc_unified      # Linux/WSL
```

On a source-based install, this updates the source in place. On WSL, it only updates the WSL side — use vbc sync wsl from Windows for the other direction. On a compiled (standalone binary) install, this refreshes the bundled source used by vbc sync wsl but not the compiled binary itself — to update a compiled install, replace it with a newly downloaded binary.

vbc sync wsl / vbc copy wsl / vbc global wsl

Push a Windows install of VBC into WSL.

```
vbc sync wsl    # copy + link, in one step
vbc copy wsl    # copy files into WSL only
vbc global wsl  # (re)link and add to PATH inside WSL only
```

If multiple WSL distros are registered, you'll be asked which one(s) to target.

vbc help

```
vbc help          # all commands
vbc help watch
vbc help plot
vbc help reset
vbc help clone
vbc help sync
```

.vbcignore

vbc init creates a .vbcignore file in your project root. Files and folders matching these patterns — and anything matched by .gitignore — are excluded from the watcher's default view and from a default vbc clone export.

```
# Directories — trailing slash
node_modules/
dist/
build/
.git/
__pycache__/
.venv/

# Wildcards
*.log
*.pyc
*.min.js
*.min.css
*.map

# Exact filenames
package-lock.json
yarn.lock
pnpm-lock.yaml

# VBC's own codebase-dump exports — never track a full copy
# of the project inside itself
*_c3_*
*_scc_*
```

Restart the watcher after editing .vbcignore — it reads the file only at startup.

Checkpoints (vbc plot ckp) still capture ignored files internally — see What's New above — so vbc clone --include .vbcignore (or .gitignore, or all) can bring them back in at export time without needing a fresh checkpoint.

VS Code Extension

Keyboard Shortcuts

Shortcut	Action
Ctrl+Shift+W	Plot WE Flag on active file
Ctrl+Shift+U	Plot US Flag on all files
Ctrl+Shift+K	Plot Checkpoint (prompts for a number)
Ctrl+Shift+G	Open VBC Graph (full tab)

VBC Graph

Open from the activity bar or Ctrl+Shift+G. The graph is a live SVG tree of your project history growing left to right:

```
ROOT ●──┬── src/ ■──┬── main.py ○──MF1+ ●──mf2+ ●──WF1 ●
        │           └── utils.py ○──WF1 ●
        └── components/ ■──App.jsx ○──mf1+ ●──WF1 ●
    ⬟ ckp1 "v1 stable"
    ⬟ ckp2 "v2 dashboard done"
```

Node	Shape	Meaning
ROOT	Large filled circle	Repository root
CKP	Gold diamond ⬟	Checkpoint — on root spine
Folder	Rounded square	Directory internal node
File	Hollow circle	File leaf — branch grows with each flag
WF / UF	Filled circle	Commit flags
MF+ / mf+ / CF / MF-	Hollow circle	March flags (color-coded)

Interactions:

- Click any flag → diff panel opens with before / after content
- Click a CKP diamond → popup: Reset project · Clone as zip · Make c3 doc
- Hover any node → tooltip with type, timestamp, stats, message

Search & Filter:

- Type in the search box → matching nodes highlight, canvas scrolls to first hit
- **Enter** to jump to the next match and cycle through all
- Click flag type buttons (WF / UF / MF+ / etc.) to show only those types
- ✕**Clear** resets everything

Command Palette

Press **Ctrl+Shift+P** and type **VBC** to access all commands: plot, reset, log, status, start/stop watcher, open graph.

How Data is Stored

VBC stores everything in .vbc/ at your project root. No database — plain text metadata plus a content-addressed blob store for actual file bytes.

```
.vbc/
├── header.json    ← { version, ckpCount, initTime, projectRoot }
├── snapshot.prt   ← line-separated JSON, one entry per flag
│                    (WF/UF/MF+/mf+/CF/MF-) — metadata + a hash
│                    reference, never inline content
├── stver.prt      ← checkpoint entries — metadata + a hash
│                    reference per file
└── objects/       ← content-addressed blob store — every
                     unique file content, stored once,
                     gzip-compressed
```

Every file is read and stored as raw bytes (never decoded as text), so images, PDFs, .docx, and executables round-trip byte-for-byte through flags, checkpoints, resets, and clones. Identical content across many checkpoints is stored exactly once, regardless of how many checkpoints reference it.

.vbcignore lives at the project root (not inside .vbc/), same convention as .gitignore — and .gitignore itself is now also respected everywhere .vbcignore is.

Design Principles

- **Zero external dependencies** — pure Node.js builtins: fs, path, zlib, os, child_process, crypto
- **Zero ceremony** — no staging area, no index, no required commit messages
- **Byte-exact** — every file type round-trips exactly, text or binary
- **Non-destructive** — reset never deletes flag history, only restores file content (and always backs up what it's about to overwrite)
- **Offline-first** — no network calls, no cloud, everything is local

Quick Reference Card

Command	Purpose
vbc init	initialize repo
vbc watch	start watcher (foreground)
vbc watch --bg / --stop / --status / --tail	daemon controls
vbc flag mf+ <file> -ms "msg"	set march flag manually
vbc plot we flag <file> -ms "msg"	commit single file → WF
vbc plot us flag -ms "msg"	commit all files → UF
vbc plot ckp as ckp<n> -ms "msg"	full snapshot → CKP
vbc reset we flag <file>	restore file to last WF
vbc reset we flag <file> -at WF2	restore file to specific WF
vbc reset us flag	restore all to last UF
vbc reset ckp -at ckp<n>	restore entire project to a checkpoint
vbc clone ckp<n> as <name>.zip	export checkpoint (default filters)
vbc clone ckp<n> as <name>.zip --include all	export checkpoint, everything

Command	Purpose
vbc clone . as <name>.zip	export live working directory now
vbc reconstruct <name>.zip as <dir>	unpack a zip
vbc construct <file>.prt as <dir>	scaffold from tree diagram
vbc log / vbc log <file>	flag history
vbc status	repo stats
vbc remove / vbc remove --force	delete .vbc/ (all history)
vbc update <path>	update CLI from new folder
vbc sync wsl / copy wsl / global wsl	Windows ↔ WSL linking
vbc help / vbc help <cmd>	help

Made by Pratham Kumar Uikey · github.com/pratham1kruk

Built for vibe coders.